```
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files

# 1. Create sample dataset
data = pd.DataFrame([
   ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
   ["Sunny", "Warm", "High", "Strong", "Warm", "Same", "Yes"],
   ["Rainy", "Cold", "High", "Strong", "Warm", "Change", "No"],
   ["Sunny", "Warm", "High", "Strong", "Cool", "Change", "Yes"]
], columns=["Sky", "AirTemp", "Humidity", "Wind",
         "Water", "Forecast", "EnjoySport"])

data.to_csv("training_data.csv", index=False)
print("Sample CSV created successfully!")

# 2. Upload CSV
print("Upload CSV or cancel to use sample data.")
uploaded = files.upload()

if uploaded:
   data = pd.read_csv(next(iter(uploaded))).astype(str)

data.columns = data.columns.str.strip()
X = data.iloc[:, :-1].values.tolist()
y = data.iloc[:, -1].str.strip().str.lower().tolist()
```

```
attributes = data.columns[:-1].tolist()
domains = [data[col].unique().tolist() for col in attributes]
n = len(attributes)

if not set(y).issubset({"yes", "no", "positive", "negative"}):
    raise ValueError("Use Yes/No or Positive/Negative labels.")

# 3. Helper functions
def covers(h, x):
    return all(a == "?" or a == v for a, v in zip(h, x))

def more_general(h1, h2):
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def minimal_specializations(h, x):
    result = []
    for i in range(n):
        if h[i] == "?":
            for v in domains[i]:
                if v != x[i]:
                    new = list(h)
                    new[i] = v
                    result.append(tuple(new))
    return result

def remove_general(H):
```

```
    return {h for h in H if not any(
        h != other and more_general(other, h) for other in H)}

def remove_specific(H):
    return {h for h in H if not any(
        h != other and more_general(h, other) for other in H)}

# 4. Initialize boundaries
S = {tuple("Ø" for _ in range(n))}
G = {tuple("?" for _ in range(n))}

# 5. Candidate-Elimination algorithm
for x, label in zip(X, y):
    x = tuple(x)

    if label in {"yes", "positive"}:
        G = {g for g in G if covers(g, x)}
        new_S = set()

        for s in S:
            if covers(s, x):
                new_S.add(s)
            else:
                new_S.add(tuple(
                    s[i] if s[i] == x[i] else
                    "?" if s[i] != "Ø" else x[i]
```

```
                for i in range(n)
            ))

        S = remove_general(new_S)
        S = {s for s in S if any(more_general(g, s) for g in G)}

    else:
        S = {s for s in S if not covers(s, x)}
        new_G = set()

        for g in G:
            if not covers(g, x):
                new_G.add(g)
            else:
                for h in minimal_specializations(g, x):
                    if any(more_general(h, s) for s in S):
                        new_G.add(h)

        G = remove_specific(new_G)
        G = {g for g in G if any(more_general(g, s) for s in S)}

    print(f"\nExample: {x} -> {label.upper()}")
    print("S boundary:", [list(h) for h in sorted(S)])
    print("G boundary:", [list(h) for h in sorted(G)])

# 6. Display final boundaries
```

```
print("\nFINAL CANDIDATE-ELIMINATION RESULTS")

print("\nSpecific Boundary (S):")
for h in sorted(S):
    print(dict(zip(attributes, h)))

print("\nGeneral Boundary (G):")
for h in sorted(G):
    print(dict(zip(attributes, h)))

if not S or not G:
    print("No consistent hypothesis remains.")

# 7. Visualize boundaries
rows = [["S"] + list(h) for h in sorted(S)]
rows += [["G"] + list(h) for h in sorted(G)]

fig, ax = plt.subplots(figsize=(11, max(3, len(rows) * 0.55)))
ax.axis("off")

if rows:
    ax.table(cellText=rows, colLabels=["Boundary"] + attributes,
             cellLoc="center", loc="center")

ax.set_title("Candidate-Elimination: Final Version-Space Boundaries")
plt.tight_layout()
```

plt.show()

```
    G boundary: [['?', '?', '?', '?', '?', '?']]

••• Example: ('Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same') -> YES
    S boundary: [['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']]
    G boundary: [['?', '?', '?', '?', '?', '?']]

    Example: ('Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change') -> NO
    S boundary: [['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']]
    G boundary: [['?', '?', '?', '?', '?', 'Same'], ['?', 'Warm', '?', '?', '?', '?'], ['Sunny', '?', '?', '?', '?', '?']]

    Example: ('Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change') -> YES
    S boundary: [['Sunny', 'Warm', '?', 'Strong', '?', '?']]
    G boundary: [['?', 'Warm', '?', '?', '?', '?'], ['Sunny', '?', '?', '?', '?', '?']]

    ================================================
    FINAL CANDIDATE-ELIMINATION RESULTS
    ================================================

    Specific boundary (S):
    {'Sky': 'Sunny', 'AirTemp': 'Warm', 'Humidity': '?', 'Wind': 'Strong', 'Water': '?', 'Forecast': '?'}

    General boundary (G):
    {'Sky': '?', 'AirTemp': 'Warm', 'Humidity': '?', 'Wind': '?', 'Water': '?', 'Forecast': '?'}
    {'Sky': 'Sunny', 'AirTemp': '?', 'Humidity': '?', 'Wind': '?', 'Water': '?', 'Forecast': '?'}
```

Candidate-Elimination: Final Version-Space Boundaries

Boundary	Sky	AirTemp	Humidity	Wind	Water	Forecast
S	Sunny	Warm	?	Strong	?	?
G	?	Warm	?	?	?	?
G	Sunny	?	?	?	?	?